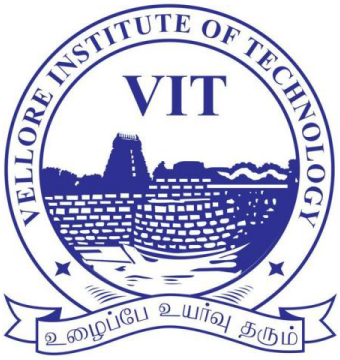
VELLORE INSTITUTE OF TECHNOLOGY, VELLORE

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING



BCSE307P – COMPILER DESIGN LAB

PROJECT REPORT — REVIEW 3

CodeTrace: Compiler-Driven Program and Security Analysis for a C Subset

Submitted by

Srikiran – 24BCE2215

Dhruv – 24BCE2211

Prince – 24BDS0153

Under the guidance of

Nithya K

Assistant Professor Senior Grade II

OCTOBER, 2026

Repository: https://github.com/lostpilot404/CD-Project

BONAFIDE CERTIFICATE

This is to certify that the project work titled **CodeTrace: Compiler-Driven Program and Security Analysis for a C Subset** is a bonafide record of work carried out by **Srikiran (24BCE2215), Dhruv (24BCE2211), Prince (24BDS0153)**, submitted in partial fulfilment of the requirements for the course BCSE307P – Compiler Design Lab, School of Computer Science and Engineering, VIT Vellore, during the academic year 2026 - 2027.

Faculty Guide: _____

Date: _____

DECLARATION

We hereby declare that the project report titled **CodeTrace: Compiler-Driven Program and Security Analysis for a C Subset** submitted by us to VIT Vellore in partial fulfilment of the requirement for the award of the degree/course BCSE307P – Compiler Design Lab is a record of bona fide work carried out by us under the supervision of Nithya K. The contents of this report, in full or in part, have not been submitted to any other institute or university for the award of any degree or diploma.

Place: _____

Date: _____

Signatures of all team members

ABSTRACT

CodeTrace is a static analysis platform for CT-C, a documented subset of C, that keeps the reasoning behind a security result visible. A hand-written scanner produces tokens with source positions, and an LR parser generator written for the project builds SLR(1), LALR(1) or canonical LR(1) tables for the same grammar. A scoped semantic analyser resolves declarations and gives each definition a stable identity; those identities drive control-flow graphs and a shared worklist framework that computes reaching definitions, liveness and definite assignment. A declarative rule language, parsed by the same LR machinery, declares sources, sinks, sanitisers and validators, and the taint analysis follows values through assignments, arguments and return values. Every finding is related to a typed program dependency graph, an evidence path and a program slice, and the whole run is exposed by a nine-view browser workbench and a documented HTTP API. On a labelled corpus of thirty-three programs the engine reports twenty expected findings with no false positive and no false negative. This report describes the system, the features completed for Review 3, and the results obtained.

TABLE OF CONTENTS

CHAPTER 1 INTRODUCTION 6
1.1 Background 6
1.2 Problem Statement 6
1.3 Objectives and Novelty 7
CHAPTER 2 EXISTING SYSTEMS AND RESEARCH GAP 8
2.1 Existing Systems 8
2.2 Research Gap 9
CHAPTER 3 PROPOSED SYSTEM 10
3.1 System Architecture 10
3.2 Module Description 10
3.3 Compiler Phases Implementation 13
CHAPTER 4 IMPLEMENTATION AND RESULTS 16
4.1 Technology and Application Workflow 16
4.2 Worked Example: Command Injection 17
4.3 Features Added in Review 3 19
4.4 Results 24
4.5 Conclusion and Future Enhancements 25
REFERENCES 26

CHAPTER 1
INTRODUCTION

1.1 Background

A compiler front end turns source text into a structured representation that later stages can read. Lexical analysis groups characters into tokens, syntax analysis decides whether the token sequence belongs to the grammar of the language, and semantic analysis resolves declarations and checks names, types and operations. These stages prepare the program for code generation, but they prepare it for program analysis as well: once a program exists as a syntax tree with resolved identifiers, values can be followed from one assignment to the next and the effect of a branch can be stated precisely [1].

Static analysis derives properties of a program without executing it. Data-flow analysis computes facts at each program point by propagating information along control-flow edges until a fixed point is reached, which tells which assignments may reach a read, which variables are live and which have certainly been assigned [4], [5]. Program dependence graphs express the same information as a graph, so a statement can be traced to what influences it and to what it influences [2], [3], [7], and slicing extracts the part of a program that can affect a chosen point [6], [8]. Security analysis applies these constructions to one question: can an attacker-controlled value reach a sensitive operation, and was it checked on the way [9], [10].

CodeTrace is built on that correspondence. It implements the classical phases for CT-C, a documented subset of C, and uses their artefacts as the basis of a taint analysis. The distinguishing property is not a new algorithm but the continuity of the evidence: the tokens, syntax tree, symbol table, control-flow graph, data-flow facts and dependency graph behind a finding are all retained and can be inspected for the same run.

1.2 Problem Statement

Tools that report security defects and tools that teach compiler construction rarely meet. A commercial scanner reports that an attacker controls a shell command and gives the result a severity, but the derivation stays inside a proprietary engine, so a reader cannot see which production or assignment made it true. A compiler course stops at printing a syntax tree.

The problem addressed here is a single system in which a security finding is derivable from the classical compiler artefacts, for a language small enough to be treated completely, and in which every step of that derivation can be inspected. Two consequences follow: the language must be bounded and documented, so a construct outside it produces a diagnostic instead of a wrong answer, and the security knowledge must be data rather than code.

1.3 Objectives and Novelty

Six objectives guided the work.

1. Implement the front end for a documented C subset: a scanner with source positions, a grammar with semantic actions, and a parser generator for SLR(1), LALR(1) or canonical LR(1) tables.
2. Resolve names with a scoped analyser that gives every declaration a stable identity, and report type and declaration errors with a code and a position.
3. Derive control-flow graphs from the syntax tree and build a reusable data-flow framework with reaching definitions, liveness and definite assignment as instances.
4. Express security knowledge in a declarative rule language parsed by the same LR machinery, with sources, sinks, sanitisers, validators and flows as declarations.
5. Relate each finding to a typed dependency graph, an ordered evidence path and a program slice, so that the result can be checked rather than trusted.
6. Present compiler and security artefacts together in a workbench and a documented API, with exports stable enough to be compared.

The novelty lies in the coupling rather than in any single component. The parser generator that accepts a CT-C program also accepts a rule file, so the security vocabulary is validated by the same machinery as the language it reasons about. Symbol identities from semantic analysis are what the taint analysis follows, so shadowing, scoping and recursion are handled once. Each dependency edge carries the result that justifies it, and the language boundary is explicit, so an absence of findings is a statement about analysed code.

CHAPTER 2
EXISTING SYSTEMS AND RESEARCH GAP

2.1 Existing Systems

Static analysis is a mature field, and the systems that exist differ in what they optimise. Compiler-based analysers reuse a production front end and are precise about the language they accept. Security scanners reuse a knowledge base of sinks and report risk at scale. Query platforms expose a program as a graph and let an analyst write the question. Teaching tools expose the phases of a compiler but not the analysis that can follow them. Table 2.1 summarises the categories that were studied and what each one leaves out for the purpose of this project.

Table 2.1: Categories of existing systems and what they leave open.

System or category	What it provides	What it leaves out here
Clang Static Analyzer, Frama-C	Production front ends with path-sensitive checks; precise diagnostics for the C family.	The derivation of a result is internal to a large codebase; extending the security vocabulary means writing C++ or OCaml plugins.
Coverity, Fortify and comparable commercial scanners	Broad rule catalogues, triage workflow and reporting for industrial codebases.	Closed engine, closed rules and no visibility into the compiler-level facts behind a finding.
Joern, CodeQL and other code-query platforms	A program represented as a graph or a database, queried by a dedicated language.	The graph is the product; the compiler phases that generate it, and the mapping from grammar to analysis, remain hidden.
ANTLR, PLY and other front-end toolkits	Grammar-driven scanner and parser construction, widely used in course projects.	Construction stops at the syntax tree; data flow, dependencies, slicing and security reasoning are left to the user.
CodeTrace	Scanner, LR table construction, semantic analysis, data flow, rule language, taint analysis, dependency graph, slicing, API and workbench in one small codebase.	Bounded to a documented C subset, with no interprocedural alias analysis and no code generation.

The comparison is not a claim of superiority. A production analyser handles a language three orders of magnitude larger than CT-C, and a commercial rule catalogue contains knowledge that this project does not attempt to reproduce. The comparison identifies what a small, complete and open implementation can offer that the large ones cannot: the whole chain from character to

finding is present in one repository and can be read in an afternoon.

2.2 Research Gap

Three concerns recur in the literature and in the tools, and none of them is fully served by the categories above. The first is explainability: a result is useful to a learner or a reviewer only if the steps that produced it are available, and most systems expose the conclusion rather than the derivation. The second is the cost of encoding knowledge: when rules are compiled into the engine, adding a sink is a programming task, and the vocabulary of the analysis cannot be inspected by the person who owns it. The third is honesty about coverage: a tool that silently ignores a construct reports an absence of defects that it cannot support.

Table 2.2 records how each concern is treated in the design, together with two further properties the rest of the report relies on, and where the evidence appears.

Table 2.2: Design concerns and their treatment.

Concern	Treatment in CodeTrace	Evidence
Explainability of a result	Every dependency edge records the analysis result that justifies it; each finding carries an ordered evidence path ending at the sink.	Sections 3.3.5, 4.2; Figure 4.1
Security knowledge as data	A declarative rule language with sources, sinks, sanitisers, validators and flows, parsed and validated before use; the built-in set is one rule file.	Sections 3.3.5, 4.3.2; Figure 4.3
Coverage that is stated	CT-C is documented construct by construct, and every construct outside it raises a diagnostic with a code, a line and a column.	Sections 3.3.1, 3.3.2; Table 4.4
Traceability across phases	Symbol identities from semantic analysis are the names the analysis follows; tokens, statements and facts share one indexing scheme.	Sections 3.1, 3.3.3
Reproducibility of a run	A run is identified by a hash of its inputs and rules; exports are schema-versioned, and two runs can be compared finding by finding.	Sections 4.1, 4.3.3; Figure 4.4

CHAPTER 3
PROPOSED SYSTEM

3.1 System Architecture

The system is organised as five layers, each of which consumes the artefact of the one before it. The front end produces tokens and a syntax tree. Semantic analysis annotates that tree with resolved symbols and types. The analysis layer builds control-flow graphs, applies the data-flow framework, performs taint propagation and constructs the program dependency graph, from which slices are taken. The reporting layer turns a completed run into findings, an evidence path and a schema-versioned document. The interface and the HTTP API present the result and accept new input. Figure 3.1 shows the layers and the artefacts that pass between them.

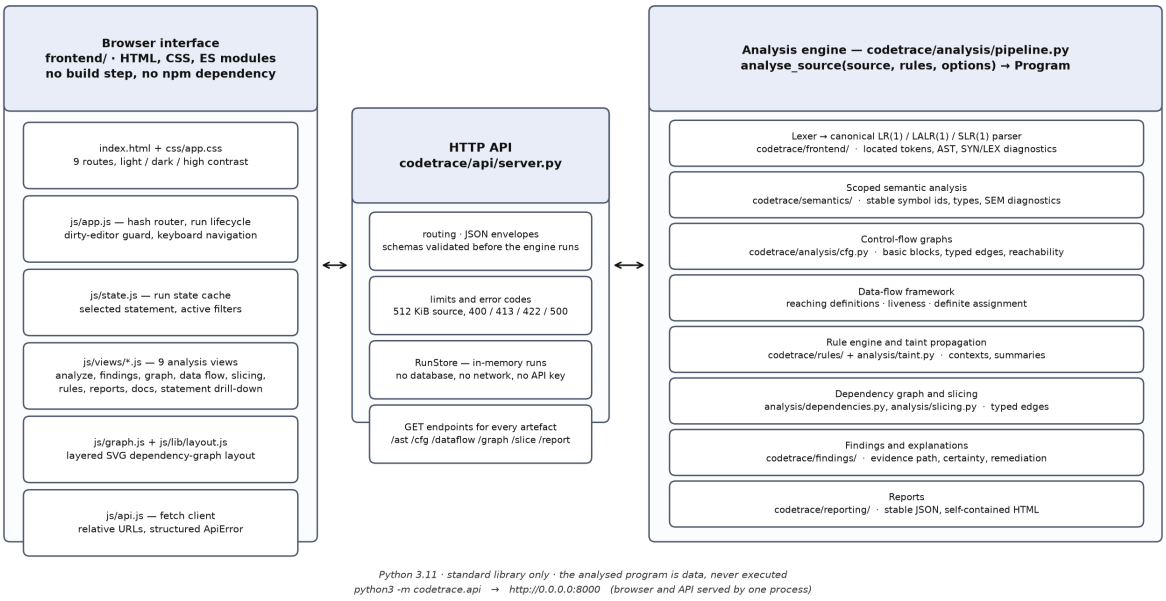


Figure 3.1: Layered architecture of CodeTrace and the artefacts passed between layers.

Two properties of the design are worth stating because the rest of the report depends on them. The pipeline is a pure function of its inputs: the source text, the selected table construction, the rule set and the analysis limits determine the result, so a run can be reproduced from its recorded configuration. And the layers communicate through documented records rather than shared mutable state, so the analysis can be executed without the interface, and the interface can be exercised against a recorded response.

3.2 Module Description

Table 3.1 lists the implementation modules with their measured size and responsibility. The engine is a Python package of seven sub-packages; the interface is a separate set of ECMAScript modules with no build step and no runtime dependency.

Table 3.1: Implementation modules, measured on the checkout.

Module	Directory	Files	Lines	Responsibility
Compiler front end	`codetrace/frontend`	9	3,277	Scanner, grammar, LR automata and tables, parser, syntax tree
Semantic analysis	`codetrace/semantics`	4	1,352	Scope construction, symbol identities, type and declaration checks
Rule language	`codetrace/rules`	4	1,235	Rule grammar, parser, validator and the built-in rule set
Program analyses	`codetrace/analysis`	11	4,130	Control-flow graphs, data-flow framework, taint, dependencies, slices
Findings and reporting	`codetrace/findings`, `codetrace/reporting`	6	1,875	Finding model, evidence paths, JSON and HTML exports
HTTP API	`codetrace/api`	4	1,152	Server, routing, request validation, catalogue endpoints
Workbench interface	`frontend`	21	4,350	Nine views, SVG graph renderer, styles, single-page shell

The same code can be read against the course. Table 3.2 maps the six modules of BCSE307L to the part of the implementation that covers each one, names the files involved, and points to the section of this report where the treatment is described. Module 6 of the syllabus, code generation, is deliberately outside the scope of an analysis tool; the table states what the implementation does provide for it.

Table 3.2: Module-wise reference against the course modules of BCSE307L.

Course module	Provided by CodeTrace	Principal files	Section
1. Introduction to compilation and lexical analysis	Hand-written scanner over the CT-C character set; tokens with kind, value, byte span, line and column; lexical diagnostics; the regular-expression specification of each token class.	`lexer.py`, `tokens.py`, `source.py`	3.3.1
2. Syntax analysis	Grammar representation with FIRST and FOLLOW sets, LR(0) item-set construction, SLR(1), LALR(1) and canonical LR(1) tables, table-driven shift-reduce parsing with error reporting.	`grammar.py`, `clr_parser.py`, `grammar_lang.py`	3.3.2
3. Semantic analysis	Syntax-directed construction of scoped symbol tables, stable symbol identities, type checking, declaration and call checks, semantic diagnostics.	`analyzer.py`, `symbol_table.py`, `externals.py`	3.3.3
4. Intermediate code generation	An annotated syntax tree as the intermediate representation, with normalised expression nodes; the typed dependency graph is the program representation the analyses consume.	`ast_nodes.py`, `dependencies.py`	3.3.4
5. Code optimization and data-flow analysis	Basic blocks and control-flow graphs, worklist framework, reaching definitions, liveness, definite assignment, loop and branch structure, slicing, security checking of the intermediate form through taint analysis.	`cfg.py`, `dataflow.py`, `taint.py`, `slicing.py`	3.3.4, 3.3.5
6. Code generation	Outside the scope of the tool. The facts a code generator needs, live ranges at every program point and the basic-block partition, are computed and exposed by the Data-Flow view; register allocation remains future work.	`liveness.py`, `cfg.py`	3.3.6, 4.5

3.3 Compiler Phases Implementation

3.3.1 Lexical Analysis

The scanner is written by hand rather than generated, because the diagnostics of the language are defined in terms of characters: an unterminated string, a preprocessor directive, a character outside the supported alphabet. Every token records its kind, its text, its byte span and its one-based line and column, and the scanner guarantees progress, so malformed input produces a diagnostic and a following token instead of a loop. Token kinds are stable strings, which lets the parser, the diagnostics and this report refer to the same names. The worked example of Section 4.2 produces forty-three tokens.

3.3.2 Syntax Analysis

The grammar of CT-C is declared once, as a list of productions with semantic actions, and the same declaration serves three purposes: it is the input of the table generator, the source of the language documentation shown in the interface, and the reference against which the grammar notes in `docs/LANGUAGE.md` are checked. The generator computes the LR(0) item sets, the FOLLOW sets and then the tables, producing the SLR(1) table, the LALR(1) table by merging item sets with equal cores, or the full canonical LR(1) table; conflicts are reported with the state, the item and the lookahead rather than resolved silently. Table 3.3 records the measured size of the automata [2], [3], [11].

Table 3.3: Parser tables for the CT-C grammar, measured at build time.

Construction	States	Productions	Conflicts	Lookahead used
SLR(1)	167	102	0	FOLLOW sets of the item sets
LALR(1), the default	167	102	0	Item sets merged on equal cores
Canonical LR(1)	656	102	0	Full lookaheads, item set per state

The LALR(1) table has the same number of states as the SLR(1) table because the grammar is SLR(1); the canonical construction is four times larger for the same language.

Parsing is table-driven: the parser holds a state stack and a value stack, consults the action and goto tables, and builds the syntax tree through the semantic actions attached to the productions that are reduced. On an error it reports the offending token, its position and the set of tokens that would have been accepted, which is the difference between a message a student can act on and one that only says that the program is wrong. Analysis of a file that fails to parse is refused rather

than attempted, so no later stage ever reasons about a partial tree. Figure 3.2 shows the pipeline as it runs for one submission.

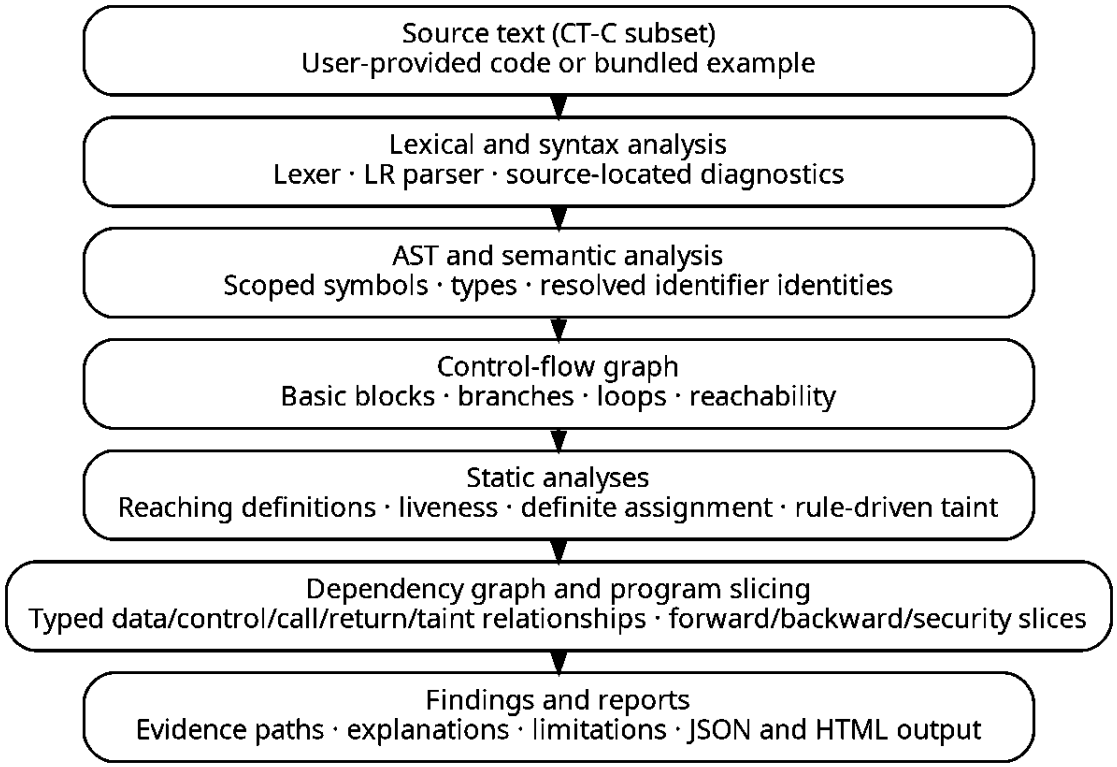


Figure 3.2: The analysis pipeline, from source text to exported report.

3.3.3 Semantic Analysis

Semantic analysis walks the tree and builds a symbol table per scope. Each declaration receives an identity made of its kind, its name and its position, so two variables that share a name in different scopes are different symbols throughout the rest of the system; this is what makes the later shadowing and recursion cases correct without special handling. The analyser checks that a name is declared before use, that types agree at assignments, conditions and returned values, that a function is called with the declared number of arguments, and that a value is not read before it is certainly assigned. The host environment is modelled explicitly: external functions such as `input`, `system` or `sql_exec` carry a declared signature, and a call to an unknown function produces a warning and keeps its argument tainted.

3.3.4 Intermediate Representation and Data Flow

The intermediate representation is the annotated syntax tree. Statements are numbered in execution order and grouped into basic blocks, and the control-flow graph records fall-through, branch, loop and return edges with the reachability of each block. The data-flow framework is a worklist solver parameterised by a transfer function and a meet operation; reaching definitions, liveness and definite assignment are three instances of it, and the number of iterations is reported with each run. Liveness yields the live range of every variable at every program point, which is the next-use information a code generator would consume.

3.3.5 Security Analysis

Security knowledge is declared in a rule file. A rule names a source, a sink, a sanitiser or a validator, gives the external function it corresponds to, the argument position concerned, the contexts in which it applies and, for a sink, a category and a severity; a flow declares which contexts a source may reach. The file is parsed by the same LR machinery as the program and validated before use, so an unknown context, a duplicate name or a flow that refers to a missing declaration is reported as a rule diagnostic. The built-in set declares five sources, eight sinks, five sanitisers, two validators and four flows, and is identified by a fingerprint that appears in every run.

The analysis follows each source value through assignments, expressions, arguments and return values, across function boundaries up to a configured recursion depth. A sanitiser clears taint only for the contexts it declares and only when it is applied before the sink; a validator guards the branch in which the value is used. Feasibility is tracked alongside the flow, so a sink in a branch whose condition is constant false is not reported, and neither is a value overwritten before the sink. Each surviving flow becomes a finding with a stable identity, a severity, a category and an ordered evidence path, and each dependency edge that carried the value records the result that justified it.

3.3.6 Code Generation

Code generation is outside the scope of the tool; what the implementation provides is the information a generator would need and the checks usually performed on the generated form. The Data-Flow view exposes basic blocks and live ranges for a chosen function, and the rule language expresses the security properties of the intermediate representation.

CHAPTER 4
IMPLEMENTATION AND RESULTS

4.1 Technology and Application Workflow

Table 4.1 lists the technology used. The engine depends on nothing outside the Python standard library, and the interface has no build step, so the whole system runs from a checkout on any machine with a Python interpreter and a browser. The source is maintained at https://github.com/lostpilot404/CD-Project.

Table 4.1: Technology used in the implementation.

Layer	Technology	Reason for the choice
Engine	Python 3, standard library only	No installation step; the lexer, parser generator, analyses and API are portable and readable without a dependency tree.
HTTP API	`http.server` with a JSON request and response schema	One process serves the interface and the API; the schema is versioned with the engine.
Interface	HTML5, CSS3, native ECMAScript modules	No bundler or framework; the workbench is served as static files by the same process.
Graphs	Hand-written SVG layout in the browser	Deterministic node placement, so the same dependency graph is drawn identically on each run and can be compared with the report.
Exports	JSON with a documented schema; self-contained HTML	A report can be archived, diffed or opened without the tool.

The application workflow follows the pipeline. A user opens the workbench, writes or loads a CT-C program, chooses the table construction and optionally supplies a rule file, and runs the analysis. The run is then available in each view: the Overview summarises what the engine measured, Findings lists the results with filters, Program Graph and Program Slicing show the dependency graph and the slices taken from it, Data-Flow Analysis shows the control-flow graph of a function with its facts, and Reports exports the run or compares it with an earlier one. The same operations are available over the API, which is what the interface itself calls.

Table 4.2 explains the work files of the repository, grouped by directory, with the size of each group measured on the checkout. The descriptions follow the docstrings of the modules, so the table and the code cannot drift apart unnoticed.

Table 4.2: Work files of the repository, measured on the checkout.

Directory	Files	Lines	Contents
Compiler front end	9	3,277	Tokenizer, grammar, LR automata and tables, parser, syntax tree
Semantic analysis	4	1,352	Scopes, symbol identities, type and declaration checks, call sites
Rule language	4	1,235	Rule parser, validator, immutable rule set, built-in rule file
Program analyses	11	4,130	Control-flow graphs, data-flow framework, taint, dependency graph, slicing
Findings and reporting	6	1,875	Finding model, evidence paths, JSON and HTML report builders
HTTP API	4	1,152	Server, routing, request validation and catalogue endpoints
Workbench interface	21	4,350	Nine views, SVG graph renderer, styles and the HTML shell
Evaluation corpus	2	754	Labels for the thirty-three programs and the harness that runs them
Example programs	34	307	The labelled CT-C programs and an example rule file
Documentation	13	4,050	Design notes, language and rule references, guides and figures
Tooling	17	10,076	Local server launch, corpus runner, figure drawing and report builders

Line counts are physical lines of source, documentation or data; binary assets are excluded. The compiler front end also carries the built-in rule file, and the interface count excludes its development fixtures.

4.2 Worked Example: Command Injection

The example below is one of the programs in the labelled corpus and is the run shown in Section 4.3. A value read from standard input is passed into a helper function, concatenated into a shell command and executed.

```
string build_command(string user_input) {
  return "ls " + user_input;
}

int main() {
  string name = input();
  string command = build_command(name);
  system(command);
  return 0;
}
```

Table 4.3 records what each phase produced, and Figure 4.1 shows the dependency graph the analysis built. The taint crosses one call boundary and three assignments; each step is recorded with the reason for the edge, which distinguishes a data dependency from a control dependency or a call edge.

Table 4.3: Phases of the run on the worked example.

Phase	Artefact for this program
Lexical analysis	43 tokens; comments skipped but counted
Syntax analysis	Accepted by the LALR(1) parser; 22 syntax-tree nodes
Semantic analysis	2 functions, 5 statements, 3 definitions, no diagnostics
Control flow	4 basic blocks, 2 control-flow edges, both functions reachable
Data flow	Reaching definitions, liveness and definite assignment reach a fixed point in 12 iterations
Taint analysis	1 flow from `input()` to `system()`, severity HIGH, certainty approximate (crosses a call)
Dependency graph	10 nodes and 10 typed edges
Report	1 finding, `FEF13A28F7E0E`, with an 8-step evidence path

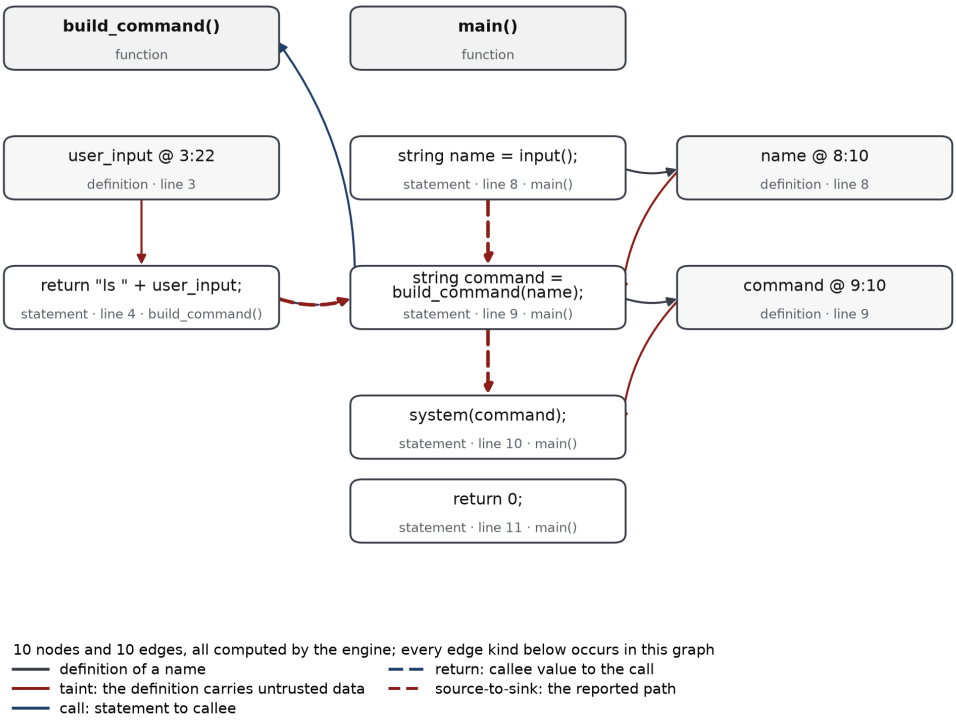


Figure 4.1: Dependency graph for the worked example: the tainted value travels from the read to the sink across one call edge and three assignments.

4.3 Features Added in Review 3

Review 2 delivered the analysis engine and the nine views. The work since then was directed at the moment a reviewer actually uses the tool: what the interface says after a run, how a rule is written and checked, how two runs are compared, and how the system explains itself. The features below are new or substantially revised in this cycle, and each is shown on the running system.

4.3.1 Guard Against Analysing Stale Text

The most damaging mistake a user of a static analyser can make is to read a result that no longer describes the text on screen. The workbench now records the text that was submitted with the current result and compares it with the editor content on every change. When the two differ, a persistent notice states that the results below belong to the submitted text and that the analysis must be run again; it is not a transient message and it disappears only when the editor matches the analysed text once more. A single deleted character is enough to raise it, and the summary below the notice is left untouched until the user runs the analysis.

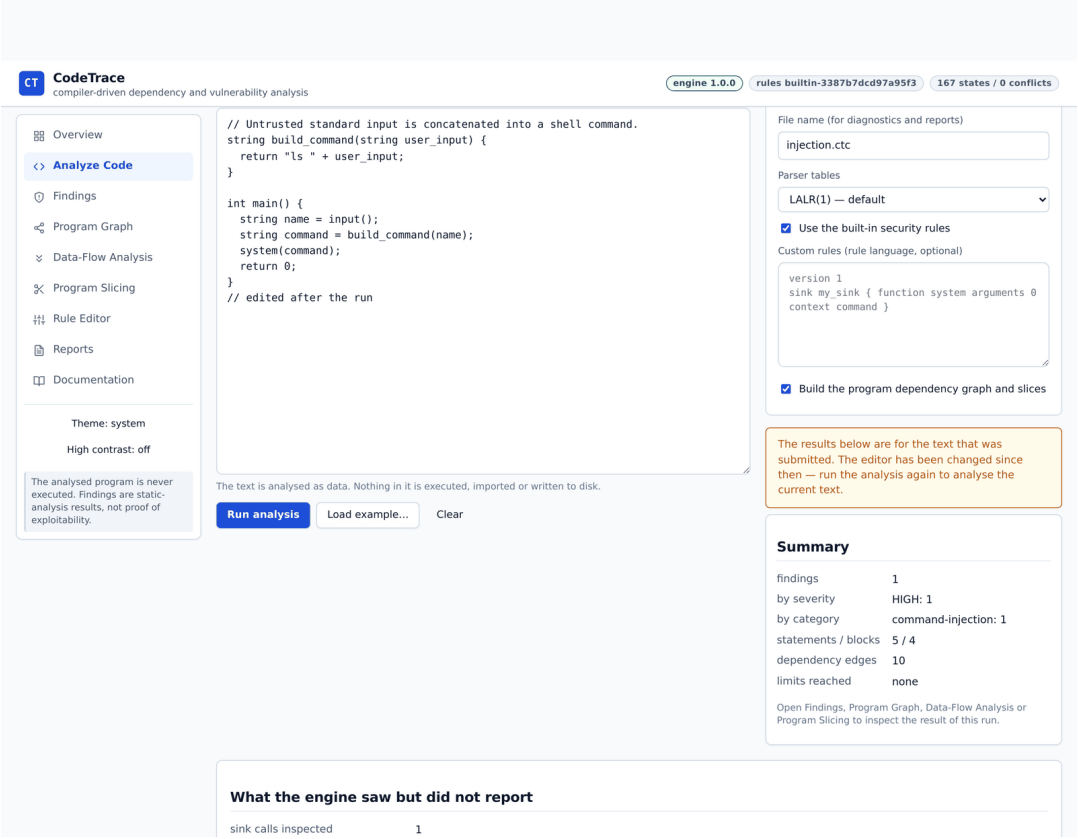


Figure 4.2: The editor modified after the run: the notice states that the summary belongs to the submitted text, and the summary is unchanged until the analysis is run again.

4.3.2 Rule Authoring and Validation in the Workbench

A rule file can be written in the workbench, validated and then used for an analysis without leaving the page. Validation reports whether the file was accepted by the rule parser, how many tokens were read, the count of each kind of declaration, and the fingerprint and version that identify the resulting rule set; a rejected file reports the position and the reason. Rules are treated as untrusted input: the parser accepts declarations only, and nothing in a rule file can execute, import or write. The built-in rules are listed beside the editor in the same vocabulary, so a new rule can be modelled on an existing one.

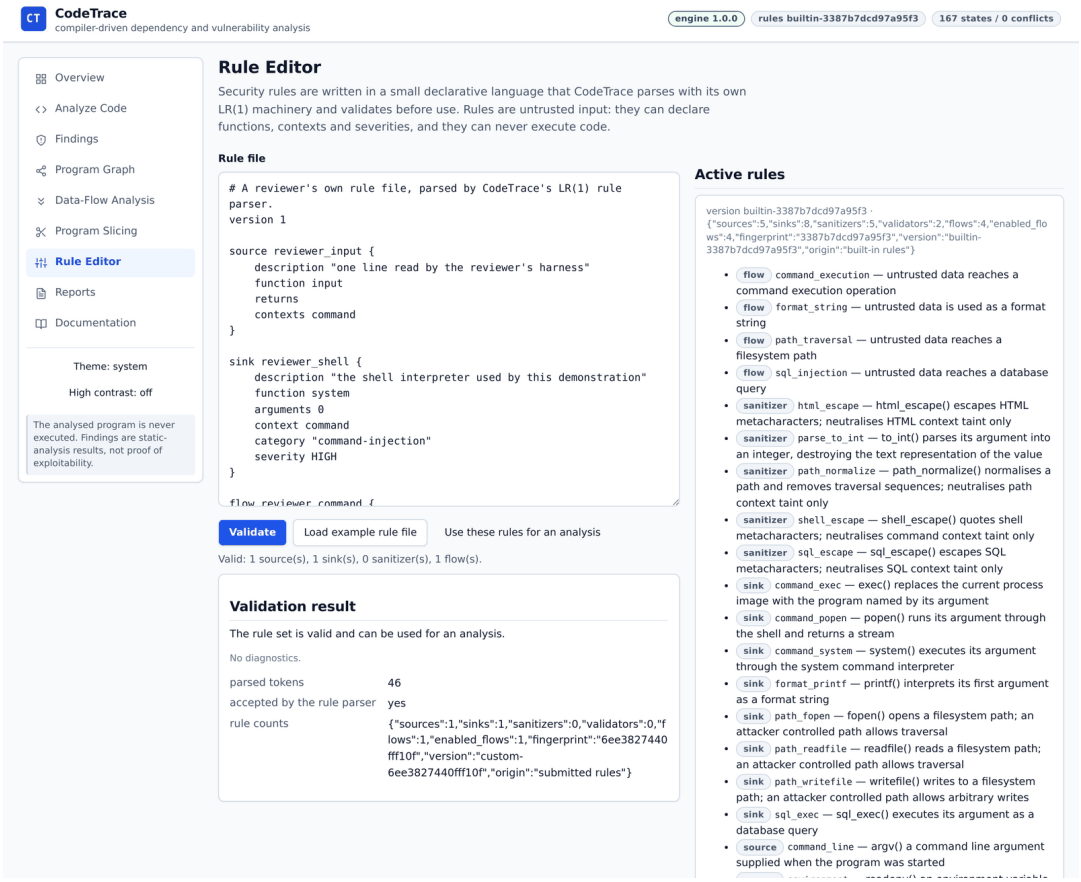


Figure 4.3: Rule Editor: a submitted rule file, the built-in rules with their declared contexts, and the validation result of the submitted file.

4.3.3 Run Comparison and Report Export

The Reports view exports the current run as a schema-versioned JSON document or as a self-contained HTML file in which the analysed source is escaped and shown in context. It also compares two runs. Findings are matched by their stable identity, derived from the rule, the sink operation and the statement text, so inserting a line above a finding does not make it new. Each finding is classified as new, resolved, persistent or moved, which makes it possible to state what a change did to the security posture of a program rather than only what the current state is.

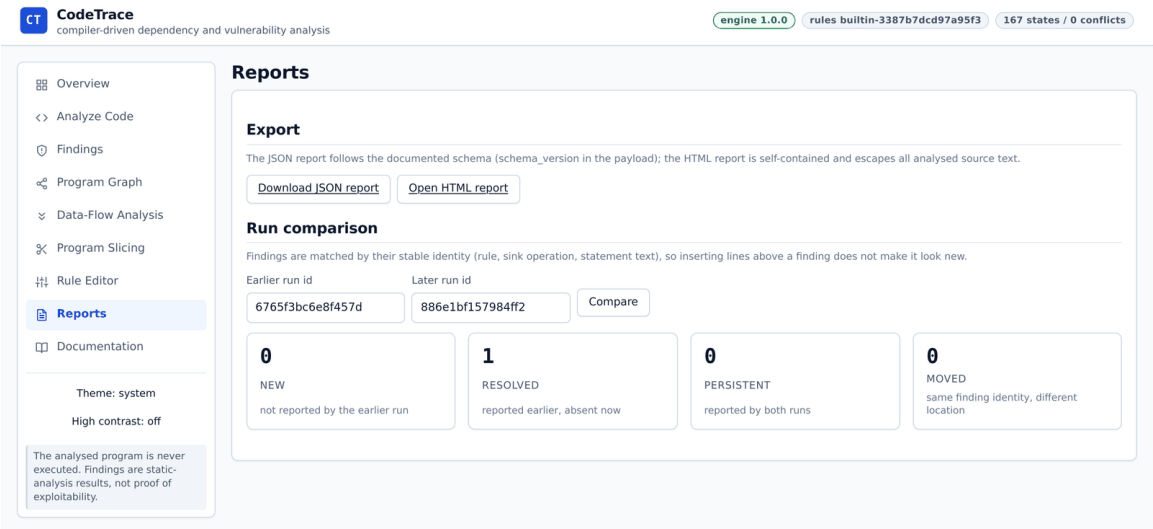


Figure 4.4: Reports: JSON and self-contained HTML export of the current run, and the comparison of two runs by finding identity.

4.3.4 Documentation Inside the Application

The Documentation view answers the questions a reviewer asks first: which constructs the language accepts, what each diagnostic code means, how the grammar is written, and which document in the repository covers a topic. The list of documents is read from the repository rather than typed into the view, so a new note appears as soon as it is added. Alongside that map the view states the types, statements and expressions the language accepts, and lists the constructs outside the subset with the reason each one is refused, which is the reference a reader needs when the analyser rejects a program.

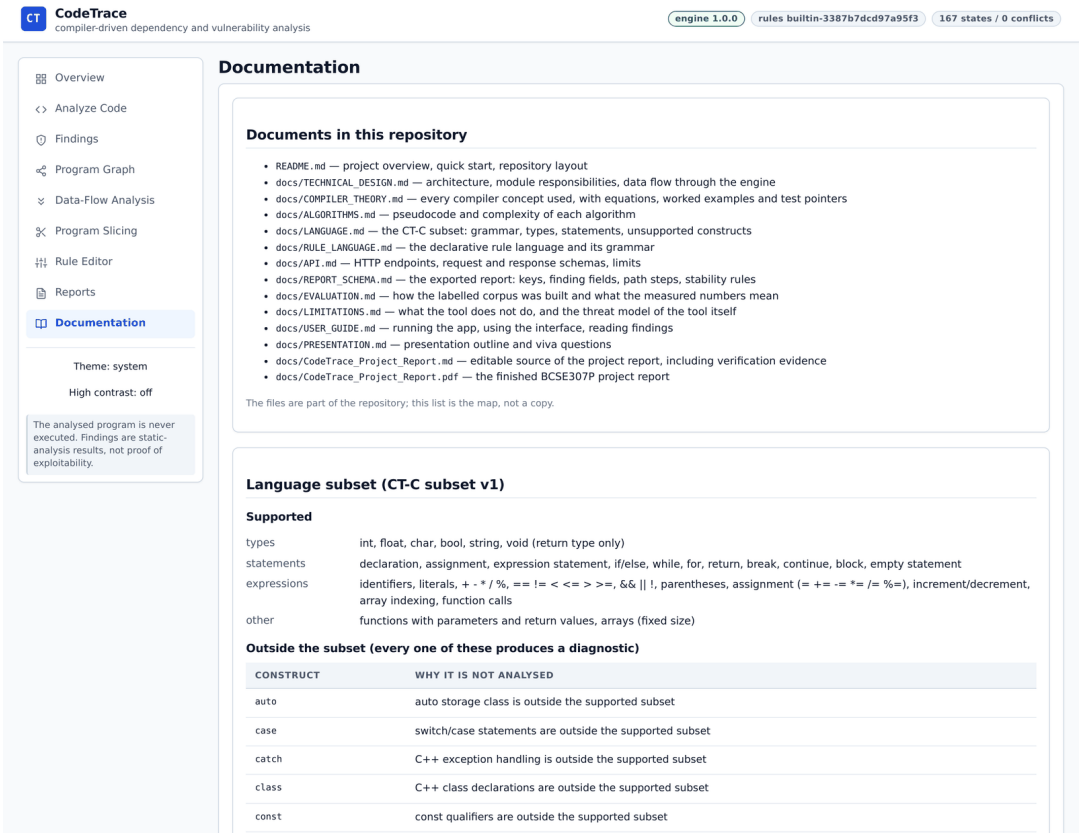


Figure 4.5: Documentation view: the documents in the repository and the constructs of CT-C, with the constructs that are rejected and the reason given for each.

4.3.5 Themes and Layout Corrections

The interface offers a light theme, a dark theme and a high-contrast variant, chosen independently of the operating system and remembered between sessions; all three are defined once, so every view inherits them. The same cycle corrected two layout defects found while capturing the interface. The notice described in Section 4.3.1 overflowed its column and overlapped the panel below it, and long values in the run summary were clipped at the panel edge. Both are visible corrected in the captures above and below: the notice in Figure 4.2, and the summary in the figure that follows, where every value is shown in full.

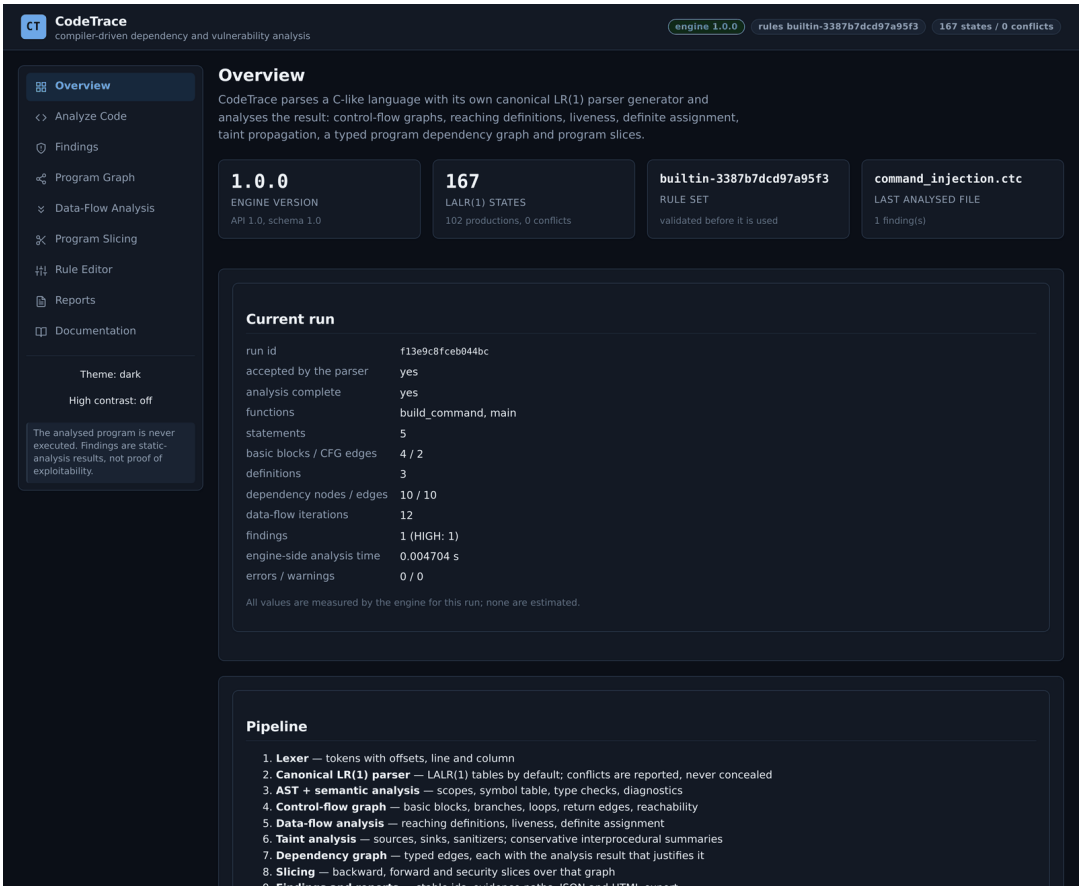


Figure 4.6: The dark theme applied to the Overview of a completed run, with the run summary shown in full.

4.4 Results

The engine was evaluated against a corpus of thirty-three CT-C programs written for this purpose, each with hand-written labels: the findings expected, their rule, category, severity, sink statement and source, and the diagnostics expected when a program is outside the language. Table 4.4 gives the outcome.

Table 4.4: Evaluation over the labelled corpus of thirty-three programs.

Group	Programs	Expected findings	Reported	Outcome
Vulnerable	17	19	19	Rule and severity as expected
Custom rules	1	1	1	Reported by the submitted file
Safe	5	0	0	No finding
Sanitised	4	0	0	No finding; taint cleared
Syntax errors and unsupported constructs	6	0	0	Rejected with the expected code
Total	33	20	20	All matched; no false positive or negative

A finding is matched to its label on rule, category, severity, sink statement and source, so precision, recall and F1 are 1.00. A program rejected before analysis counts as correct when the expected diagnostic is reported.

The corpus is small, and the result is evidence that the implementation behaves as specified on the cases it was designed for, not a measurement of detection rate on real code. The cases probe shadowing, loop-carried flow, a sanitiser applied in the wrong context, a validator guarding the other branch, decoys in comments and strings, and interprocedural flow. Constructs outside CT-C are rejected, not approximated.

4.5 Conclusion and Future Enhancements

The project set out to make the reasoning behind a static analysis result inspectable for a language small enough to be treated completely. The classical phases are implemented rather than imported: the scanner, the grammar, the three LR table constructions, the parser and the semantic analyser are in the repository, and the later analyses build on their output. The security layer is data parsed by the same machinery, and each finding carries its path and its edges.

The Review 3 work added what makes the system usable rather than only demonstrable: the guard against stale results, rule authoring and validation, comparison of two runs, in-application documentation and a readable set of themes. On the labelled corpus the engine reports every expected finding and rejects every program outside the language.

Four directions remain open: a field-sensitive alias analysis, without which pointers and structs cannot be added; function summaries, so that the interprocedural analysis scales beyond the current recursion limit; learning the rule set from labelled programs; and a code generator driven by the live ranges the framework computes.

REFERENCES

[1] A. V. Aho, M. S. Lam, R. Sethi and J. D. Ullman, *Compilers: Principles, Techniques and Tools*, 2nd ed. Boston: Pearson Education, 2007.

[2] D. E. Knuth, "On the translation of languages from left to right," *Information and Control*, vol. 8, no. 6, pp. 607-639, 1965.

[3] F. L. DeRemer, "Simple LR(k) grammars," *Communications of the ACM*, vol. 14, no. 7, pp. 453-460, 1971.

[4] G. A. Kildall, "A unified approach to global program optimization," in *Proceedings of the ACM Symposium on Principles of Programming Languages*, 1973, pp. 194-206.

[5] F. Nielson, H. R. Nielson and C. Hankin, *Principles of Program Analysis*. Berlin: Springer, 1999.

[6] M. Weiser, "Program slicing," *IEEE Transactions on Software Engineering*, vol. 10, no. 4, pp. 352-357, 1984.

[7] J. Ferrante, K. J. Ottenstein and J. D. Warren, "The program dependence graph and its use in optimization," *ACM Transactions on Programming Languages and Systems*, vol. 9, no. 3, pp. 319-349, 1987.

[8] T. Reps, S. Horwitz and M. Sagiv, "Precise interprocedural dataflow analysis via graph reachability," in *Proceedings of the ACM Symposium on Principles of Programming Languages*, 1995, pp. 49-61.

[9] B. Chess and G. McGraw, "Static analysis for security," *IEEE Security and Privacy*, vol. 2, no. 6, pp. 76-79, 2004.

[10] MITRE Corporation, "Common Weakness Enumeration: CWE Top 25 most dangerous software weaknesses," 2024. [Online]. Available: https://cwe.mitre.org/top25/

[11] D. Watson, *A Practical Approach to Compiler Construction*. Cham: Springer International Publishing, 2017.